

Ferramenta de Processamento de Imagens

Vetor de Mat's com Interface Tkinter

Aluno: Franciney

Disciplina: Processamento de Imagens / Visão Computacional

Arquivo: ferramenta_imagens.py

1. Objetivo

Desenvolver uma ferramenta interativa que gerencia um **vetor (lista) de imagens** (Mat's do OpenCV) e permite aplicar, visualizar e comparar **5 categorias** de processamento:

- 1 Conversão de cor
 - 2 Filtro
 - 3 Detector de borda
 - 4 Binarização
 - 5 Morfologia matemática
-

2. Arquitetura

```
.....
·                               App (tk.Tk)                               ·
.....
·  Lista de      ·  Preview: Original · Processada      ·
·  Imagens      ·                               ·
·  (Listbox)     · .....
·                               Métodos: Dropdown + Slider +      ·
·  [Adicionar]   ·                               Botão "Aplicar"   ·
·  [Remover]     ·                               ·
·  [Limpar]      ·  [Salvar Resultado]      ·
.....
```

Vetor de Mat's `self.vetor_imagens: list[ImagemProcessada]`

Cada elemento contém:

- `mat_original` · imagem original (np.ndarray / Mat)
 - `mat_processada` · resultado após aplicar um método
 - `metodo` · descrição do último método aplicado
-

3. Métodos Disponíveis

| Categoria | Métodos OpenCV | Método Manual ({}|-----|-----|-----|) | · Cor | Cinza, HSV, Lab, YCrCb, HLS | · Cinza-Manual | Filtro | Gaussiano, Mediana, Bilateral, Cartoon | · Média-Manual | Borda | Canny, Laplaciano, Sobel X/Y | · Sobel-Manual | Binarização | Otsu, Adaptativo, Global | · Otsu-Manual | Morfologia | Abertura, Fechamento, Erosão, Dilatação, Gradiente | · Erosão-Manual |

4. Implementações Manuais

4.1 · Cinza-Manual · Conversão de cor

O que faz: Converte BGR para escala de cinza sem usar `cv2.cvtColor`.

Fórmula (ITU-R BT.601):

$$Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

Por que esses pesos? O olho humano é mais sensível ao verde, por isso ele tem o maior peso (0.587).

Código:

```
def _cinza_manual(self, img_bgr):
    b = img_bgr[:, :, 0].astype(np.float64)
    g = img_bgr[:, :, 1].astype(np.float64)
    r = img_bgr[:, :, 2].astype(np.float64)
    cinza = (0.114 * b + 0.587 * g + 0.299 * r).astype(np.uint8)
    return cinza
```

4.2 · Média-Manual · Filtro passa-baixa

O que faz: Suaviza a imagem aplicando a média dos vizinhos em uma janela $k \times k$.

Princípio Cria um kernel onde cada peso = $1/(k^2)$ e faz convolução 2D.

Exemplo kernel 3×3 :

```
.....
. 1/9  1/9  1/9  .
. 1/9  1/9  1/9  .
. 1/9  1/9  1/9  .
.....
```

Código:

```
def _media_manual(self, img, k):
    kernel = np.ones((k, k), dtype=np.float32) / (k * k)
    return cv2.filter2D(img, -1, kernel)
```

4.3 · Sobel-Manual · Detector de bordas

O que faz: Detecta bordas calculando o gradiente da imagem nas direções X e Y.

Kernels definidos manualmente:

Gx (bordas verticais):

```
.....
. -1   0   1   .
. -2   0   2   .
. -1   0   1   .
.....
```

Gy (bordas horizontais):

```
.....
. -1  -2  -1   .
.  0   0   0   .
.  1   2   1   .
.....
```

Magnitude: $M = \sqrt{G_x^2 + G_y^2}$

Código:

```
def _sobel_manual(self, gray):
    kx = np.array([[ -1,  0,  1], [ -2,  0,  2], [ -1,  0,  1]],
dtype=np.float32)
    ky = np.array([[ -1, -2, -1], [ 0,  0,  0], [ 1,  2,  1]],
dtype=np.float32)
    gx = cv2.filter2D(gray, cv2.CV_64F, kx)
    gy = cv2.filter2D(gray, cv2.CV_64F, ky)
    magnitude = cv2.magnitude(gx, gy)
    return cv2.convertScaleAbs(magnitude)
```

4.4 · Otsu-Manual · Binarização

O que faz: Encontra automaticamente o melhor limiar para separar fundo e objeto, sem usar `cv2.THRESH_OTSU`.

Algoritmo:

- 1 Calcular o histograma da imagem (256 valores)
- 2 Para cada limiar t de 0 a 255:
 - Separar pixels em duas classes (fundo e objeto)
 - Calcular a **variância entre classes**
- 3 O limiar que **maximiza** a variância é o ideal

Código resumido:

```
def _otsu_manual(self, gray):
    hist = cv2.calcHist([gray], [0], None, [256], [0, 256]).ravel()
    # Para cada t: calcula variância = peso_bg × peso_fg × (média_bg -
média_fg)2
    # Retorna o t que maximiza a variância
    resultado = np.zeros_like(gray)
    resultado[gray > melhor_limiar] = 255
    return resultado
```

4.5 · Erosão-Manual · Morfologia matemática

O que faz: Para cada pixel, verifica a vizinhança $k \times k$ e atribui **valor mínimo**

Efeito: Encolhe regiões brancas e remove ruídos pequenos.

Princípio:

```
Pixel de saída = MIN(vizinhança k×k)
Exemplo 3×3:
.....
. 200 180 190 .
. 210 220 205 .   . resultado = min(todos) = 180
. 195 215 200 .
.....
```

Código:

```
def _erosao_manual(self, img, k):
```

```
h, w = img.shape[:2]
pad = k // 2
padded = cv2.copyMakeBorder(img, pad, pad, pad, pad,
                             cv2.BORDER_CONSTANT, value=255)

resultado = np.zeros_like(img)
for i in range(h):
    for j in range(w):
        vizinhanca = padded[i:i + k, j:j + k]
        resultado[i, j] = vizinhanca.min()
return resultado
```

5. Como Executar

`.\venv\Scripts\python.exe ferramenta_imagens.py`

Dependências OpenCV, NumPy, Pillow, Tkinter (nativo do Python).

6. Fluxo de Uso

- 1 **Adicionar** imagens à lista (suporta múltiplas)
 - 2 **Selecionar** uma imagem · aparece no painel "Original"
 - 3 **Escolher** método no dropdown (com · = implementação manual)
 - 4 **Ajustar** parâmetros com o slider
 - 5 **Aplicar** · resultado aparece lado a lado
 - 6 **Salvar** o resultado processado em PNG/JPG/BMP
-

7. Conclusão

- A lista Python (`list[ImagemProcessada]`) substitui o **vector de Mat's** do C++
- Cada categoria tem ao menos **1 implementação manual**(·) para demonstrar domínio dos algoritmos
- A interface Tkinter permite experimentação rápida e comparação visual
- Os sliders proporcionam ajuste de parâmetros **em tempo real**